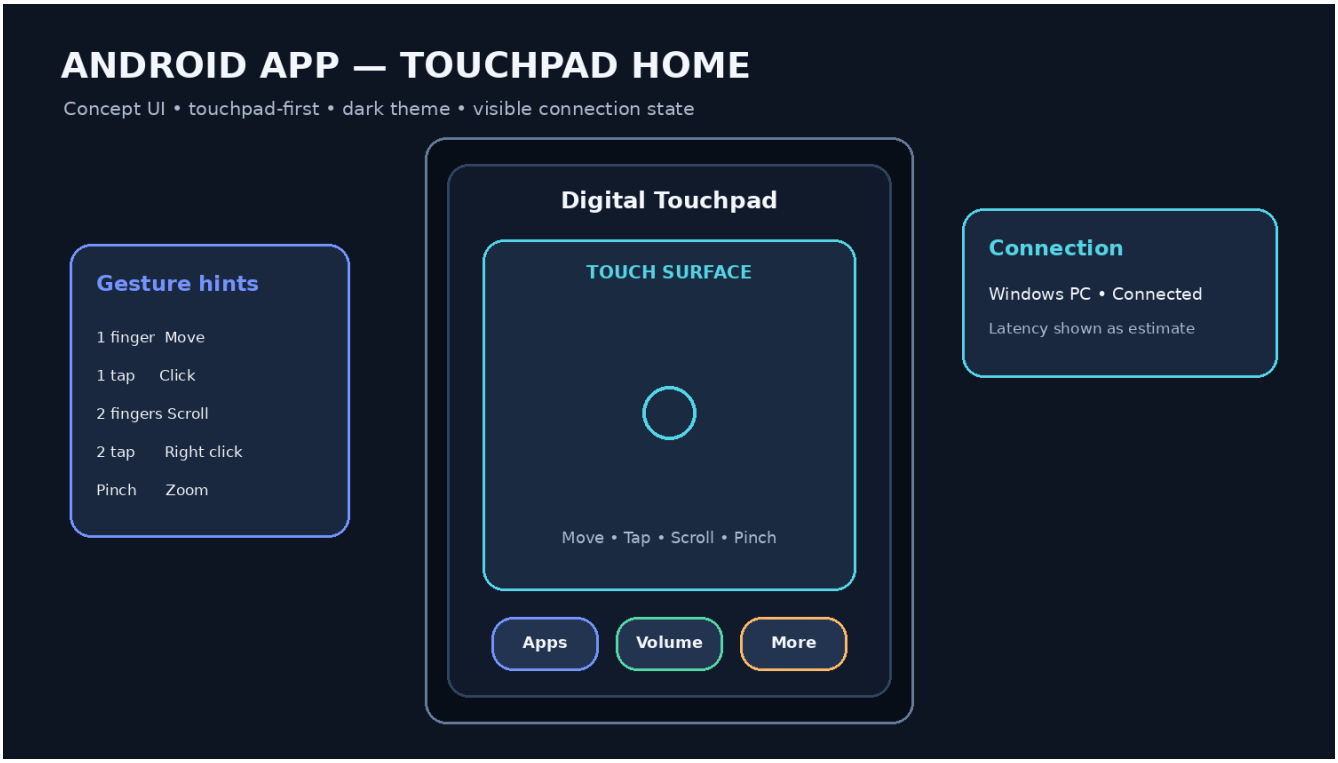
PRODUCT REQUIREMENTS DOCUMENT

Digital Touchpad

Android phone as a wireless touchpad and control surface for Windows laptops and PCs



Version: 2.0 **Date:** 5 November 2026 **Status:** Product definition baseline

Scope: Android mobile app + Windows companion receiver; Wi-Fi receiver mode and Bluetooth HID mode.

Audience: Product, UX, Android, Windows, QA, security and release teams.

This is a build-oriented product contract. Proposed defaults are to be validated during prototyping; they are not claims of existing implementation.

Document map and conventions

Sections	Coverage
01–04	Executive summary, goals, scope and user journeys
05–08	Experience, connection modes, gestures and settings
09–12	Quick actions, receiver, architecture and protocol
13–16	Security, compatibility, diagnostics and NFRs
17–20	Acceptance tests, roadmap, risks and decisions
Appendices	Terminology, traceability and references

• MUST = release-blocking unless explicitly deferred; SHOULD = expected; MAY = optional.

• P0 = MVP critical; P1 = important for first stable release; P2 = later enhancement.

• Windows laptops and desktops are the first release target. Interactive Android classroom boards are a future platform track, not part of Windows MVP.

Version	Date	Change
1.0	Initial	Initial concept
1.1	3 Nov 2026	Expanded product definition
2.0	5 Nov 2026	Detailed Windows-first specification, UI concepts, gestures, architecture, security, diagnostics and QA

01. Executive summary

Digital Touchpad turns an Android phone into a portable touchpad and remote-control surface for a Windows laptop or desktop. The phone provides a large touch-sensitive area, multi-touch gestures and a compact dock of system and application actions. Windows receives standard pointer/keyboard input through either direct Bluetooth HID or a secure local-network connection to a Windows companion receiver.

Wi-Fi + Windows Receiver is the preferred primary path because it enables richer actions, pairing management, diagnostics and future extension. **Bluetooth HID** is a receiver-free alternative for basic mouse/keyboard-style input when the phone and OEM Bluetooth stack support the HID Device role.

The product must feel responsive, predictable and safe. It must never silently execute arbitrary commands, leave mouse buttons or keys held after disconnect, or claim an action succeeded when Windows rejected it.

Product promise: Your phone becomes a precise, familiar control surface for your Windows PC—without carrying a mouse.

Product principles

• Touchpad first: pointer control is the primary screen and fastest path to use.

• Two connection paths: Wi-Fi for rich control; Bluetooth for receiver-free basic input.

• Predictable gestures: documented, configurable and conflict-aware.

• Local-first: Wi-Fi works without internet when devices can reach each other on the LAN.

• Safe by design: authenticated pairing, allowlisted actions and explicit destructive-action confirmation.

• Honest compatibility: show what a device can do; never promise unsupported hardware features.

02. Users, needs and success outcomes

User	Need	Outcome
Student / everyday user	Control PC from couch, bed or away from desk	Reliable pointer, click, scroll and media control
Presenter / lecturer	Navigate slides and apps while away from laptop	Quick actions, show desktop, app switching
Power user	Tune gestures and shortcuts	Profiles, configurable mappings, low latency
IT / support	Diagnose pairing and compatibility issues	Clear health state, safe logs and tests

Jobs to be done

• When away from my laptop, use my phone as a touchpad instead of reaching for a mouse.

• When presenting, access slides, apps and media without a remote desktop interface.

• When Bluetooth HID is unavailable, use Wi-Fi receiver mode for richer control.

• When something fails, identify whether the cause is pairing, network, receiver, permission or Windows capability.

Proposed success metrics

Metric	Proposed target	Measurement
First-time Wi-Fi pairing	≥95% on supported test matrix	Pairing funnel / lab tests
Pointer latency	P95 ≤50 ms on healthy LAN	Timestamped client and receiver
Input reliability	≥99.5% test actions delivered once	Automated action sequences
Stuck input defects	0 known reproducible cases	Crash / disconnect test suite
Crash-free sessions	≥99.5%	Opt-in crash diagnostics
Basic task completion	≥90% without help	Usability study

Targets are proposed engineering goals, not guaranteed network performance.

03. Scope and release boundaries

In scope — Windows-first

• Android touchpad, gesture engine, connection setup, quick-action dock and settings.

• Windows companion receiver with signed standalone installer and portable distribution.

• Wi-Fi mode with QR/PIN pairing and authenticated encrypted local session.

• Bluetooth HID mode for standard mouse/keyboard-style reports on validated Android devices.

• Pointer, primary/secondary/middle click, drag, scroll, zoom fallback and configurable mappings.

• Allowlisted Windows shortcuts, media controls, volume, lock and supported power actions.

• Connection status, capability detection, input tests and redacted diagnostic export.

• Onboarding, permission education, updates and recovery flows.

Out of scope for Windows MVP

• Remote desktop, screen streaming, file transfer or viewing the PC screen on phone.

• Arbitrary shell, PowerShell, command-line or script execution.

• Unrestricted remote administration or public-internet access to receiver.

• Guaranteed control of elevated apps or UAC secure desktop.

• Native Windows Precision Touchpad multi-contact emulation over Bluetooth.

• Built-in Android classroom-board control and iOS client.

Release	Deliverables
MVP / R1	Wi-Fi receiver, pairing, pointer, click, drag, scroll, core shortcuts, volume/media, diagnostics, installer
R1.1	Bluetooth HID on validated phones, expanded gestures, brightness capability checks, portable receiver polish
R1.2	Profiles, custom allowlisted app shortcuts, presenter mode, richer diagnostics
Future	Interactive Android board receiver, managed deployment, additional OS targets

04. Core user journeys

A. First-time Wi-Fi setup

Step	User action	System behaviour
1	Install app and Windows receiver	Receiver starts; displays network/firewall state
2	Open pairing screen	Receiver displays QR and short-lived numeric code
3	Scan QR or enter code	Phone validates receiver identity and shows PC name
4	Approve on Windows	User explicitly approves and may name phone
5	Bind trust	Exchange credentials; store trusted relationship
6	Connect	Encrypted session; show Connected and latency
7	Test	Pointer/click test; explain any blocked input

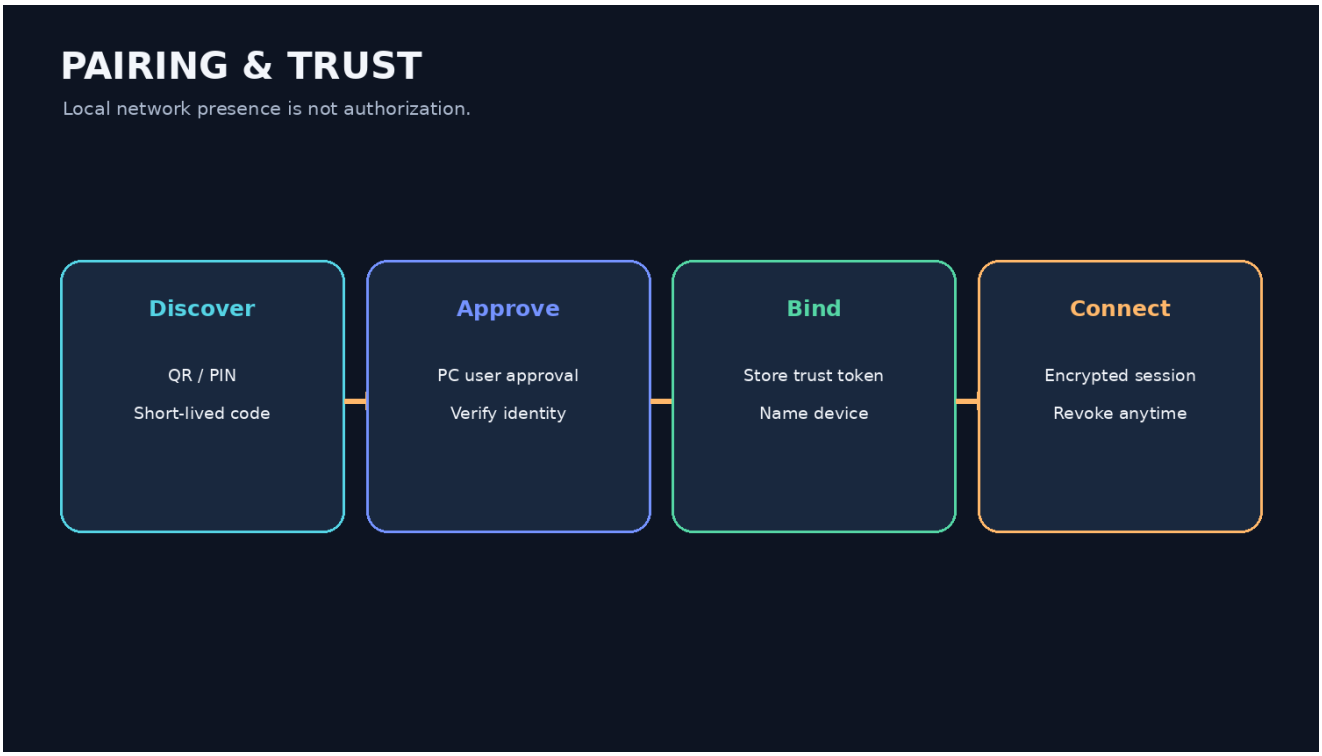


Figure 1. Pairing lifecycle: LAN discovery is not authorization.

B. Everyday control

• App shows last trusted PC and last-known status.

• If reachable and trusted, connect; optional auto-reconnect is configurable.

• Touch surface activates only after authenticated session is ready.

• User moves pointer, clicks, scrolls and uses dock actions.

• On disconnect, show clear state; receiver releases all held inputs.

C. Bluetooth direct mode

• User selects Bluetooth HID and follows Android pairing guidance.

• App registers HID Device profile if supported and displays capability state.

• User pairs from Windows Bluetooth settings; app tracks host connection.

• Basic pointer/buttons and supported keyboard/media reports are available; Wi-Fi-only actions are marked unavailable.

D. Recovery

• Pause outgoing gestures on loss and release any held state.

• Classify cause where possible: peer unreachable, receiver stopped, auth rejected, Bluetooth lost or app backgrounded.

• Offer one primary recovery action: reconnect, open receiver, re-pair or run diagnostics.

• Use bounded reconnect backoff; never flood the network.

05. Product experience and navigation

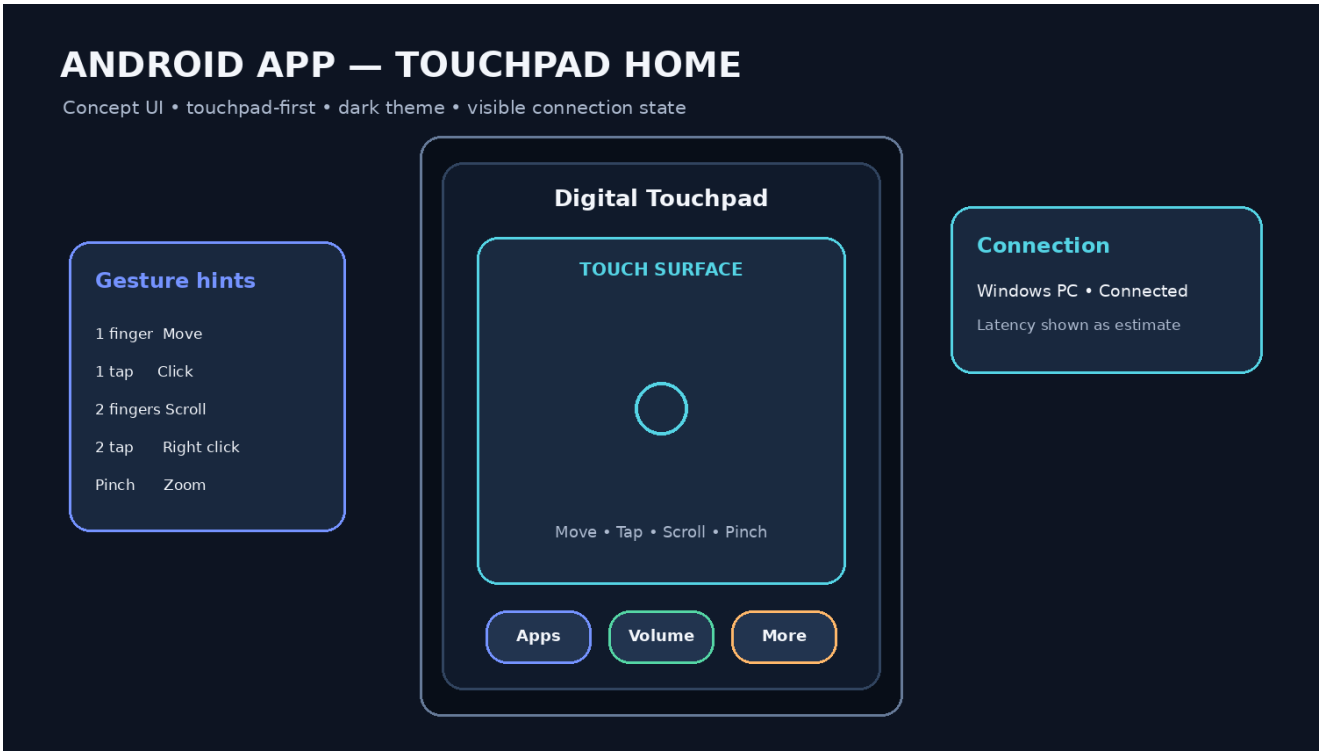


Figure 2. Android touchpad home concept; status values are illustrative.

Area	Purpose	Core UI
Touchpad	Primary control	Large pad, status, click buttons, gesture hint
Quick actions	Windows shortcuts	Apps, Search, Task Manager, screenshot, desktop, lock
System controls	Media and hardware	Volume, mute, playback, brightness if supported, power
Connections	Manage PCs and modes	Wi-Fi/Bluetooth, paired PCs, QR, reconnect
Gesture settings	Tune interaction	Mappings, sensitivity, scroll, edge controls
Diagnostics	Troubleshoot	Health, input tests, receiver version, export
Settings	Preferences	Theme, haptics, startup, privacy, permissions

Touchpad screen requirements

• Touch surface dominates screen; controls do not crowd the active pad.

• Compact connection status shows PC name and selected transport.

• Optional explicit left/right buttons for users preferring button controls.

• Optional subtle haptic click feedback; configurable.

• Portrait-first layout; landscape may use side dock.

• Handle Android safe insets and system navigation conflicts.

• First-use gesture coach can be dismissed and reopened.



Figure 3. Windows receiver concept: tray-first, explicit trust and health state.

Windows receiver UX

• System-tray-first with compact management window.

• States: Ready, Connected, Pairing, Disconnected, Blocked and Error.

• Show paired devices, revoke, startup preference, version and diagnostics.

• Portable build explains user-level limitations and firewall setup.

06. Connection modes

Mode A — Wi-Fi + Windows receiver (primary)

Phone and PC communicate over local network using an authenticated encrypted session. This works without internet if devices can reach each other on the LAN. It supports richer commands and future extension.

Requirement	Definition
Discovery	QR or short pairing code; optional mDNS; manual IP fallback
Transport	WebSocket over TLS or equivalent authenticated encrypted channel
Network	Same LAN or reachable hotspot; client isolation/firewall can block traffic
Authentication	One-time pairing approval, per-device credential, session auth and revocation
Input	Pointer delta, button state, scroll and allowlisted key/action commands
Recovery	Heartbeat, timeout, reconnect with backoff, release-all on close
Offline	No internet dependency for normal local control

Mode B — Bluetooth HID (secondary)

Android's BluetoothHidDevice API is available from API level 28, but API availability does not guarantee that a phone/OEM firmware supports or reliably exposes the HID Device role. Runtime capability checks and real-device validation are mandatory. Android API reference.

Requirement	Definition
Role	Phone behaves as standard Bluetooth HID peripheral; Windows is host
Receiver	No Windows receiver required for basic mouse input
Reports	Standard mouse buttons and relative pointer; optional keyboard/media reports

Requirement	Definition
Pairing	Windows Bluetooth settings; app reports state
Advanced actions	No custom receiver commands; unavailable unless expressible through HID
Limit	No guarantee of native Windows Precision Touchpad multi-contact reports

- Default onboarding recommends Wi-Fi for full feature set.
- Offer Bluetooth only after runtime capability check; explain unsupported devices.
- Do not silently switch transports mid-session.
- Store preferred mode per PC; allow manual override.

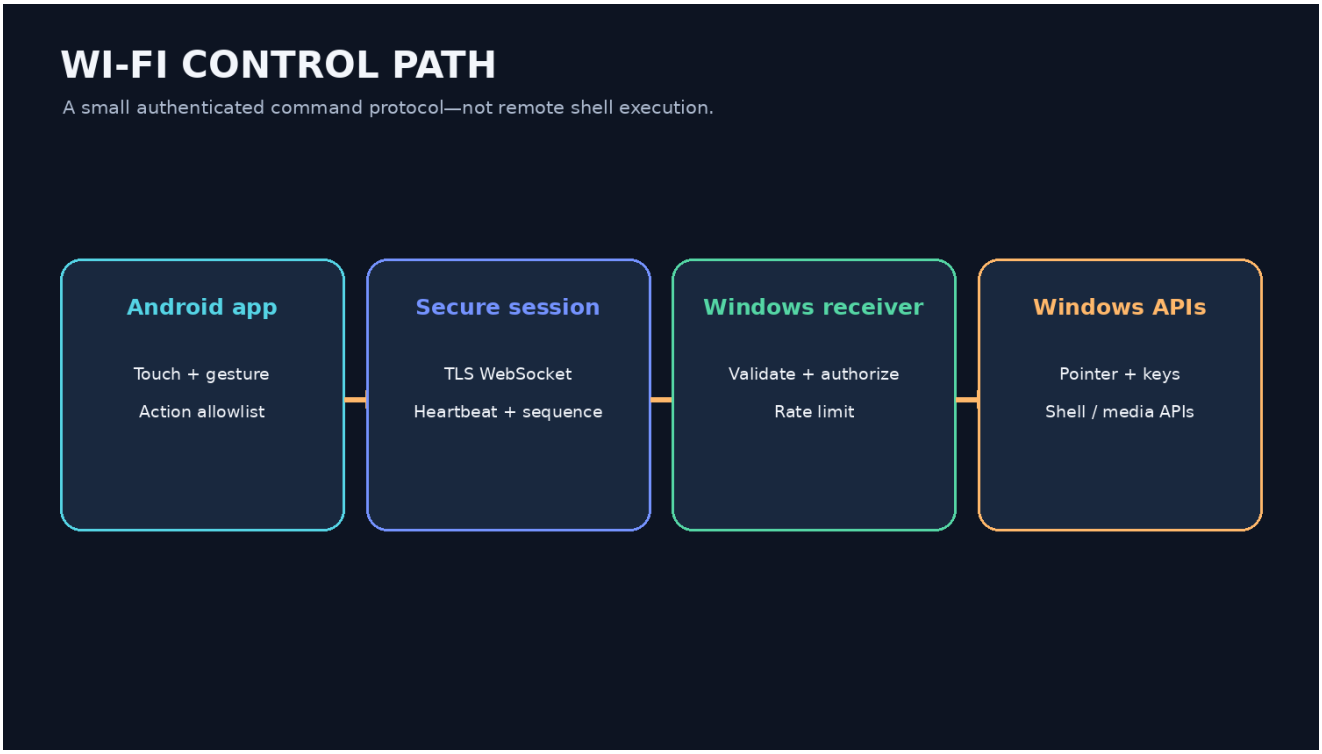


Figure 4. Wi-Fi command path and validation boundaries.

07. Pointer and gesture requirements

Gesture recognition is a first-class subsystem. Raw contacts must be translated into stable commands. A click is not emitted until the recognizer rules out movement, drag, multi-touch and cancellation.

ID / Priority	Requirement	Acceptance behaviour
IN-001 P0	One-finger pointer	Continuous relative movement; stops on lift
IN-002 P0	Single tap	One primary click within time/movement thresholds
IN-003 P0	Double tap	Two qualifying taps in interval produce double click, not duplicate singles
IN-004 P0	Tap-hold drag	Hold engages left-down; move drags; lift releases
IN-005 P0	Explicit buttons	Optional left/right controls generate correct down/up
IN-006 P0	Two-finger tap	One secondary click
IN-007 P1	Three-finger tap	Middle click; configurable
IN-008 P0	Release safety	Release all held buttons/keys on cancel, pause, disconnect and shutdown

Gesture	Default mapping	Notes
Two-finger vertical	Wheel scroll	Speed and direction configurable
Two-finger horizontal	Horizontal wheel	Fallback configurable
Pinch in/out	Ctrl + wheel zoom fallback	App-specific support varies

Gesture	Default mapping	Notes
Three-finger swipe up	Win+Tab / Task View	Shortcut mapping
Three-finger swipe down	Win+D / Show Desktop	Shortcut mapping
Three-finger left/right	Alt+Tab-style app switch	Controlled key sequence
Four-finger left/right	Win+Ctrl+Left/Right	Virtual desktops; version/settings dependent
Four-finger up/down	Configurable	No default until user research

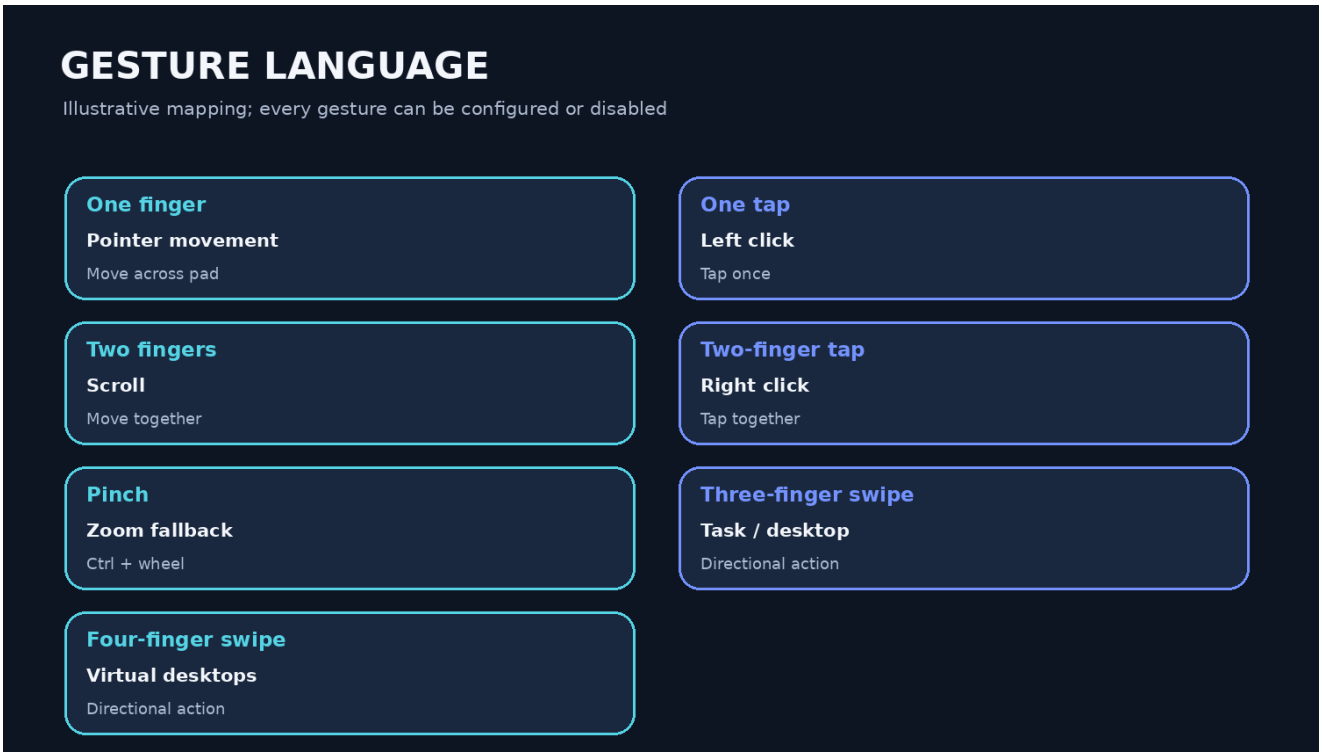


Figure 5. Gesture language overview.

Proposed recognizer defaults

Parameter	Starting value	Tuning
Tap duration	≤180 ms	User test; accessibility adjustment
Tap movement	≤10 dp	Scale by density
Double tap	≤300 ms	Configurable 220–450 ms
Hold to drag	≥350 ms	Visual drag state
Multi-touch settle	≤80 ms	Prevent false single tap
Swipe distance	≥80 dp	Scale to pad size
Cancel	Any touch cancel / focus loss	Release all immediately

These are initial tuning values, not immutable OS standards. Final values must be established through prototype testing.

08. Gesture settings and conflict management

- Sensitivity, scroll speed, natural/inverted direction and horizontal-scroll toggle.
- Enable/disable tap-to-click, double-tap, drag, two-finger tap and three-finger tap.
- Per-direction three-/four-finger swipe mappings and action picker.
- Pinch mapping via Ctrl+wheel fallback or custom key combination.
- Gesture presets: Standard, Presenter, Accessibility-friendly and Custom.
- Optional edge gestures for volume/brightness MUST be off by default.
- Multi-finger recognition suppresses one-finger click for that gesture sequence.
- Touch cancellation or focus loss triggers release-all before new gesture processing.

• Unsupported transport mappings show unavailable; never silently remap.

• Provide test mode showing recognized gesture and mapped action before execution.

09. Quick actions and system controls

Quick Actions are explicit shortcuts, not diagnostics. Every action has a stable allowlisted ID, capability state, permission and confirmation rule.

Quick action	Default implementation	Priority / guardrail
Settings	Open Windows Settings via supported URI	P0; capability check
File Explorer	Fixed Explorer launch target	P0
Windows Search	Win+S	P0
Task Manager	Ctrl+Shift+Esc	P0
Terminal	Configured Windows Terminal; else unavailable	P1
Calculator	Calculator app	P1
Browser	Default browser; no arbitrary URL by default	P1
Screenshot	Win+Shift+S	P1
Show Desktop	Win+D	P0
Lock PC	Win+L	P1; explicit tap
Custom app	Receiver-managed allowlist	P2; no arbitrary command/path

System control	Behaviour	Limit
Volume	Master volume step/continuous	Supported Windows audio API
Mute	Toggle mute	State feedback if readable
Media	Play/pause/next/previous	Active media session varies
Brightness	Internal display brightness if exposed	External monitor may require DDC/CI
Lock	Lock workstation	Explicit tap, no extra confirm
Sleep / hibernate	Confirmation sheet	Policy/power settings may block
Restart / shutdown	Confirmation + 10s countdown + cancel	User configurable; deduplicate request

Power action safety

• Sleep, hibernate, restart and shutdown require second explicit confirmation.

• Restart/shutdown display target action, countdown and cancel.

• Countdown remains cancellable until OS request submission.

• Unique request ID prevents duplicate power execution.

• If policy blocks action, show unavailable with reason where known.

Optional touchpad-edge swipe for volume/brightness is disabled by default and only enabled after opt-in. It must not interfere with normal pointer movement.

10. Windows receiver requirements

• Provide signed standalone installer and portable version.

• Installer supports install, repair, update and uninstall; clean up product-created firewall/startup entries on uninstall.

• Prefer user-level execution; elevate only for justified setup/configuration.

• Start-on-login is opt-in; tray-first background operation.

• Receiver orderly shutdown releases all held inputs.

• Show version, Windows version, status, paired devices, revoke, startup, firewall guidance and diagnostics.

• Accept pairing only after local user approval.

• Validate strict schema, action allowlist, rate limits, sequence and replay checks.

• Expose only required local-network listener; no public internet listener.

• Logs redact sensitive data by default.

Windows input implementation

Wi-Fi pointer input may use Windows SendInput. Microsoft documents that SendInput is subject to UIPI: a process can inject input only into applications at an equal or lower integrity level. A blocked call may not provide a reliable error that identifies UIPI as the cause. Therefore elevated windows and secure desktop prompts are not guaranteed control targets. Microsoft SendInput documentation.

• Dedicated serialized input dispatcher.

• Track held buttons/keys per session; release on abnormal termination.

• No raw executable paths, shell strings or scripts accepted from phone.

• App launching resolves receiver-owned allowlisted IDs locally.

• Do not bypass UAC or secure desktop.

11. Technical architecture

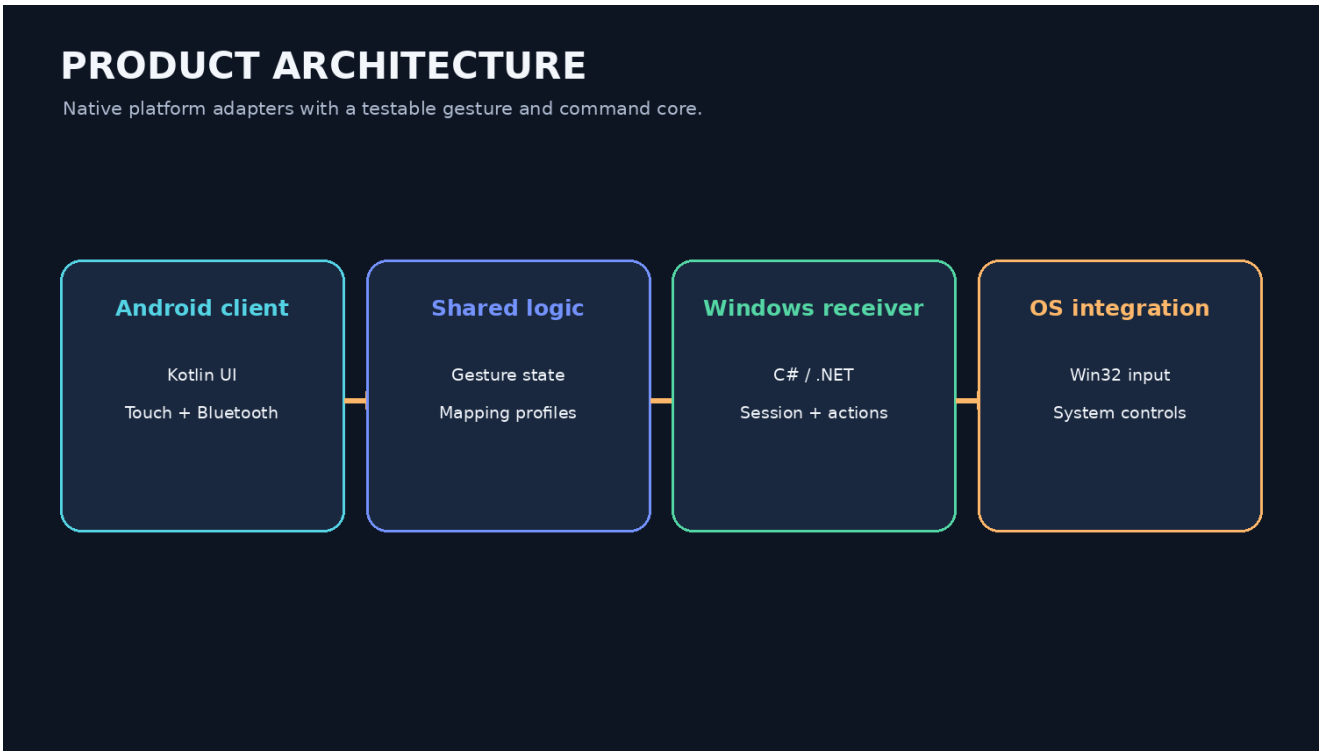


Figure 6. Recommended high-level architecture.

Component	Recommended stack	Reason
Android app	Kotlin + Jetpack Compose	Native HID, permissions, lifecycle and haptics
Touchpad UI	Compose pointer input / custom view	Multi-touch and low-latency surface
Gesture core	Kotlin module initially	Testable state machine; avoid premature cross-language sharing
Windows receiver	C# / .NET tray app	Windows API and installer ecosystem
Input integration	Win32 SendInput + supported shell/media APIs	Standard OS path with documented UIPI limit
Wi-Fi protocol	WebSocket over TLS	Bidirectional session and heartbeat
Discovery	QR + manual IP; optional mDNS	Managed networks may block multicast

Component responsibilities

- Android UI: touch, visual feedback, navigation, settings and consent.
- Gesture engine: contact classification, thresholds, state machine and mapping profiles.
- Transport adapters: Bluetooth HID reports or authenticated Wi-Fi messages.
- Windows session manager: pairing, authentication, heartbeat, replay protection and cleanup.
- Action dispatcher: schema validation, capability check, allowlist and execution.
- Diagnostics: health signals and user-consented redacted report.

Connection state model

State	Meaning	Transitions
Unpaired	No trusted host	Pairing
Pairing	Short-lived setup	Paired / Failed / Cancelled
Paired / Offline	Trust exists; peer unreachable	Connecting / Revoked
Connecting	Handshake underway	Connected / Failed
Connected	Authenticated session	Degraded / Disconnected / Revoked
Degraded	High latency/loss	Connected / Disconnected
Disconnected	Session ended; input released	Connecting / Offline
Revoked	Trust removed	Pairing

12. Wi-Fi protocol and command contract

Use a versioned JSON or compact binary envelope over authenticated WebSocket. This is a control protocol, not a remote command interpreter. Every message has type, version, session ID, monotonic sequence and request ID for discrete actions.

Field	Purpose	Rule
version	Compatibility	Major mismatch rejects; minor negotiation
type	Message class	Enumerated values only
session_id	Bind to session	Unpredictable; invalid after disconnect
sequence	Ordering/replay	Monotonic per direction
request_id	Idempotency	Unique per discrete action
payload	Typed fields	Strict schema and bounded values

Message	Direction	Purpose
PAIR_REQUEST / APPROVE	Both	Trust after explicit approval
SESSION_HELLO / READY	Both	Version/capability negotiation
POINTER_MOVE	Phone → PC	Relative dx/dy, bounded and rate-limited
BUTTON_STATE	Phone → PC	Button down/up
SCROLL	Phone → PC	Wheel deltas
KEY_CHORD	Phone → PC	Allowlisted key codes/chords
ACTION_REQUEST / RESULT	Both	Execute allowlisted action and report outcome
HEARTBEAT	Both	Liveness and latency
RELEASE_ALL	Either	Clear held input
CAPABILITIES	PC → phone	Advertise supported actions

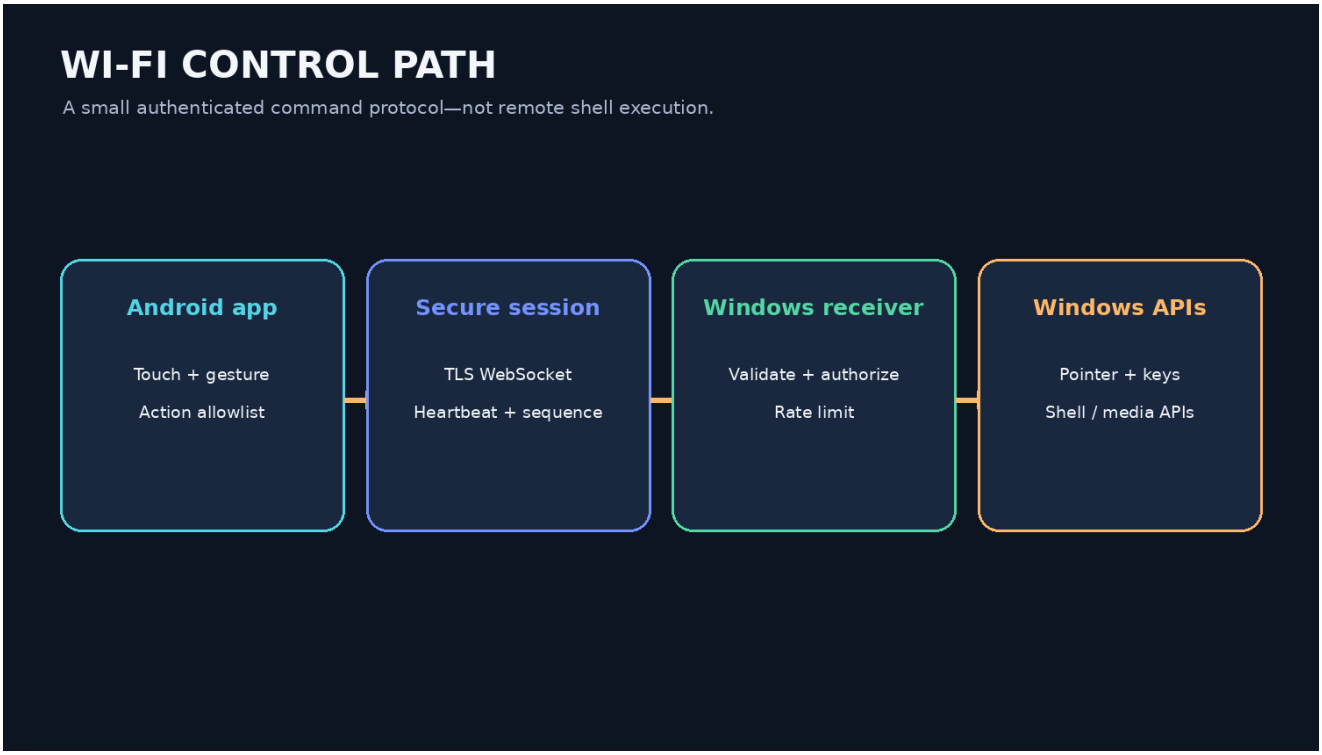


Figure 7. Wi-Fi command path.

Validation and network defaults

• Bound pointer/wheel deltas, message size and command frequency.

• Reject malformed types, unknown action IDs, out-of-range values and stale sequence numbers.

• Check authorization and capability before execution.

• Return accepted / executed / unsupported / denied / failed distinctly.

• No free-form shell command or executable path in any payload.

Parameter	Proposed default
Heartbeat	2 seconds
Disconnect timeout	6 seconds without valid heartbeat
Reconnect backoff	1, 2, 4, 8 seconds; cap 30 seconds with jitter
Pointer update rate	Up to 120/s; coalesce when congested
Action timeout	5 seconds default; action-specific
Pairing code	2-minute lifetime; one-time use
Paired devices	10 per receiver initially

13. Security, privacy and trust

Threat model assumes an untrusted device may be on the same Wi-Fi. Network presence alone must never authorize input injection. Protect against unauthorized pairing, replay, tampering, accidental power actions and log leakage.

• Pairing requires short-lived code and explicit approval on PC.

• Use TLS with authenticated peer identity; bootstrap certificate trust/pinning securely.

• Bind credentials to device and receiver; use Android Keystore and Windows protected storage where feasible.

• Support revoke-one and revoke-all.

• Use session nonces, sequence numbers and replay protection.

• Bind listener to intended local interfaces; do not expose WAN/cloud listener.

• No arbitrary shell, PowerShell, file browsing or remote code execution.

• Allowlist actions and validate every parameter.

• Power actions require confirmation and idempotency.

• Release held input on every failure path.

Privacy

• Never capture screenshots, clipboard, typed text, passwords, browser content or keystroke history.

• Diagnostics collect only technical metadata; user consent required before export.

• Redact local IPs, device IDs and personal paths by default.

• Explain privacy during onboarding; analytics optional and opt-in.

Abuse case	Mitigation
Unknown phone sends input	Pair approval + authenticated session
Replay shutdown request	Unique request ID + sequence + expiry
Malformed pointer flood	Schema validation + rate limits + bounded deltas
Receiver exposed beyond LAN	Interface binding + firewall guidance + no cloud relay
Stuck button after crash	Watchdog + release-all on timeout
Malicious app path	Receiver-managed local allowlist

14. Compatibility and platform constraints

Area	Requirement / caveat
Android baseline	Android 10+ proposed for Wi-Fi MVP; final minimum after QA
Bluetooth	HID Device support varies by phone/OEM; API 28 is not a device guarantee
Android 12+	Handle Bluetooth runtime permissions correctly
Windows	Windows 10 22H2 and Windows 11 proposed; validate releases
UIPI	Lower-integrity receiver may not inject into elevated targets
Secure desktop	UAC secure desktop/sign-in screens out of scope
Brightness	Internal panel varies; external monitor may require DDC/CI
Power	Sleep/hibernate depends on hardware, policy and settings
Media	Active media session behaviour varies
Firewall	Network profile and rules can block local listener
Virtual desktops	Shortcuts depend on Windows version/settings

Network conditions

• Same Wi-Fi LAN is simplest supported case.

• Phone hotspot supported if Windows joins hotspot and client isolation is off.

• Guest/campus/enterprise Wi-Fi may block peer-to-peer traffic.

• VPNs and multiple adapters may route/block traffic; diagnostics identify likely causes.

• No internet is required once LAN reachability exists.

15. Diagnostics and support

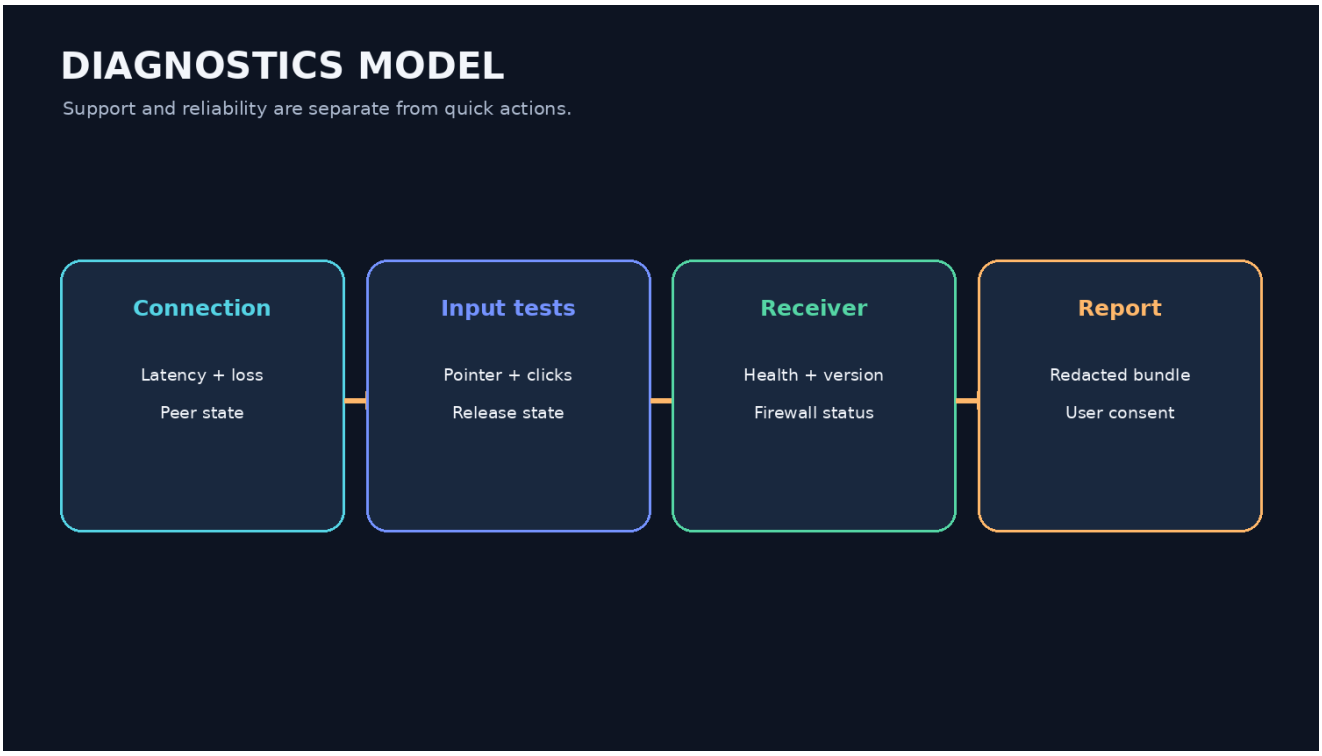


Figure 8. Diagnostics are a distinct support feature, not a quick-action category.

Category	Signals / tests	User output
Connection	Transport, peer reachability, latency, loss, heartbeat age	Connected / Degraded / Offline + next step
Input	Pointer, left/right/middle click, scroll, release	Pass/fail per test
Receiver	Process, version, listener, firewall profile	Running / blocked / outdated
Bluetooth	HID capability, registration, host connection	Supported / unsupported / disconnected
Capabilities	Volume, media, brightness, power	Available / limited / unavailable
Export	Version, OS, transport, redacted errors	Consent-based bundle

- One-tap quick test verifies connection, pointer, click and receiver health.
- Latency is a recent estimate, not a guaranteed performance claim.
- Never record actual keystrokes, text entry or screen contents.
- Preview diagnostic categories and allow cancellation before export.
- Troubleshooting must match detected failure category.

16. Non-functional requirements

Area	Requirement
Latency	P95 pointer command ≤50 ms target on healthy LAN; Bluetooth benchmark separately
Responsiveness	Touch feedback immediate; network calls never block UI
Reliability	Reconnect without duplicate discrete actions; release held inputs on all disconnects
Security	Authenticated encrypted session; strict allowlist; no arbitrary remote execution
Availability	Receiver starts/recover reliably after network changes
Battery	Coalesce pointer updates; avoid constant high-power scanning
Accessibility	Scalable text, accessible labels, adjustable thresholds, high contrast
Localization	Externalized strings and locale-aware formatting
Maintainability	Versioned protocol, structured logs, testable gesture state machine
Updates	Signed installer/update artifacts and recovery guidance
Usability	First successful connection possible without reading long manual

Performance test conditions

- 2.4 GHz and 5 GHz Wi-Fi, hotspot, congested LAN and client-isolated network.
- Idle, pointer-heavy, scrolling and simultaneous shortcut use.
- Median, P95/P99 latency, packet loss and reconnect time.
- Screen locked, app backgrounded, battery saver and receiver minimized.
- Separate transport latency from Windows scheduling/rendering latency.

17. Acceptance criteria and QA

ID	Test	Pass criteria
AT-001	Fresh Wi-Fi pairing	PC approval required; unapproved phone cannot control
AT-002	Pointer	Continuous movement; no runaway after lift
AT-003	Single/double click	Exactly expected click count
AT-004	Drag	Hold engages; release ends in every tested path
AT-005	Right/middle click	One mapped click; disabled mapping does nothing
AT-006	Scroll	Direction/speed match settings
AT-007	Pinch fallback	Ctrl+wheel/configured mapping works in test apps
AT-008	Gesture conflict	Multi-touch never causes unintended single click
AT-009	Disconnect safety	Kill app/receiver, Wi-Fi off, PC sleep; no held input
AT-010	Quick actions	Expected target or clear unavailable result
AT-011	Power	Confirmation, countdown and cancel work
AT-012	UIPI	Elevated target limitation handled honestly
AT-013	Bluetooth	Unsupported phone identified; validated phone pairs
AT-014	Revocation	Revoked phone cannot reconnect without pairing
AT-015	Malformed traffic	Schema, replay and rate attacks rejected
AT-016	Diagnostics	Consent; no keystrokes/screenshots/content
AT-017	Packaging	Install, launch, update/repair, uninstall and portable tested

Test matrix

Dimension	Coverage
Android	Minimum/current OS; Android 12+ permissions
Phone vendors	At least 3 OEM families; HID support verified per model
Windows	Windows 10 22H2 and Windows 11 supported builds
Network	Home router, hotspot, guest/client isolation, managed-like LAN
Displays	Internal panel, external monitor, mixed DPI multi-monitor
Power	Sleep/resume, hibernate if enabled, shutdown/restart
Security	Standard/elevated app, UAC, firewall blocked, revoked phone

18. Roadmap and delivery plan

Phase	Deliverables	Exit gate
0 — Spikes	HID proof on sample phones; Wi-Fi secure pairing; SendInput proof; installer spike	Feasibility risks measured
1 — Core MVP	Android touchpad, Wi-Fi receiver, pointer/click/drag/scroll, pairing, secure protocol, core shortcuts	All P0 tests pass
2 — Reliability	Reconnect, diagnostics, release-all watchdog, installer/portable, firewall guidance	No stuck-input defects; stable matrix
3 — Advanced	Bluetooth HID, multi-finger mapping, media/volume, brightness checks, power confirmation	P1 tests pass
4 — Polish	Profiles, presenter mode, custom allowlisted shortcuts, localization/accessibility	Usability targets met
Future	Interactive Android board support, after board-specific feasibility	Separate approval

MVP cut line: Ship a high-quality Wi-Fi pointer experience with click, drag, scroll, secure pairing, release safety, core shortcuts and diagnostics first. Do not delay the first usable release for every advanced gesture.

19. Risks and mitigations

Risk	Impact	Mitigation
Bluetooth varies by OEM	Direct mode unavailable on some phones	Runtime check, tested-device matrix; Wi-Fi primary
Campus Wi-Fi blocks peers	Wi-Fi cannot connect	Manual IP, hotspot guidance, diagnostics
UIPI blocks elevated input	Some windows not controllable	Document; no bypass; user guidance
External brightness unsupported	Inconsistent control	Capability detection; DDC/CI only when validated
Gesture ambiguity	Accidental actions	State machine, thresholds, test mode
App killed/backgrounded	Session/HID lost	Lifecycle handling, reconnect, release safety
LAN security exposure	Unauthorized control	Pairing, TLS, allowlist, replay protection
Power action mistakes	Data loss/disruption	Second confirmation, countdown, cancel, deduplication
Windows API differences	Action inconsistency	Compatibility layer and OS test matrix

Dependencies

- Android Bluetooth HID and OEM Bluetooth behaviour.
- Windows input, shell, audio, power and display APIs.
- Code signing and installer distribution.
- Firewall rules and local network topology.
- QA access to representative Android phones and Windows hardware.

20. Open decisions

Decision	Recommended default	When to settle
Minimum Android	10+ for Wi-Fi MVP; Bluetooth separately	Phase 0
Windows baseline	10 22H2 + Windows 11	Before MVP freeze
Bluetooth scope	Mouse first; keyboard/media after proof	Engineering spike
Gesture thresholds	Proposed defaults; tune via user tests	Prototype
Auto reconnect	On for last trusted device; configurable	MVP
Auto-start receiver	Off until user opts in	Security review
Custom shortcuts	Receiver-managed allowlist	R1.2
Analytics	Minimal and opt-in	Privacy review
Installer signing	Signed release builds	Public beta

Non-goals reaffirmed: no screen mirroring, remote desktop, unrestricted file access, arbitrary command execution, cloud relay, Android-board control or iOS in this Windows-first specification.

Appendix A. Terminology

Term	Meaning
HID	Human Interface Device, such as mouse or keyboard
Bluetooth HID Device	Phone acts as Bluetooth peripheral to Windows host
Receiver	Windows companion app accepting authenticated Wi-Fi commands
UIPI	Windows privilege isolation limiting lower-integrity input injection
LAN	Local area network; internet not required if peers can reach each other
mDNS	Local discovery protocol; may be blocked on managed networks
DDC/CI	Monitor control protocol supported by some external displays
Release-all	Safety operation releasing all held buttons and keys
Allowlist	Explicit set of action IDs permitted by receiver

Appendix B. Requirement traceability

Area	Requirements	Test coverage
Connection	Wi-Fi, Bluetooth, pairing, reconnect	AT-001, 013, 014
Pointer / gestures	Movement, clicks, drag, scroll, pinch, swipes	AT-002–008
Actions	Apps, search, Task Manager, media, power	AT-010, 011
Receiver	Lifecycle, installer, portable, firewall	AT-009, 017
Security	Auth, allowlist, replay, release-all	AT-001, 009, 014, 015
Diagnostics	Health, tests, redacted export	AT-016
Compatibility	OEM, Windows, network, UIPI	AT-012, 013 + matrix

Appendix C. Technical references

• Android Developers — BluetoothHidDevice API: developer.android.com

• Microsoft Learn — SendInput and UIPI: learn.microsoft.com

• Microsoft Learn — HID API overview: learn.microsoft.com

The API and compatibility details must be rechecked against target SDKs and Windows releases during implementation. All diagrams and UI illustrations are original conceptual visuals, not screenshots of a working product.

End of Digital Touchpad PRD v2.0